

EEE522 Autonomous Vehicles

Localization and Mapping (Part I)

Ivan Ho

Acknowledgements

Part of the teaching materials in this subject was prepared based on the course F1Tenth at the University of Pennsylvania (UPenn). We would like to acknowledge Prof. Rahul Mangharam and his team for sharing the materials with us.

Part of the teaching materials in this chapter was prepared based on the online lecture of Prof. Cyrill Stachniss at the University of Bonn.

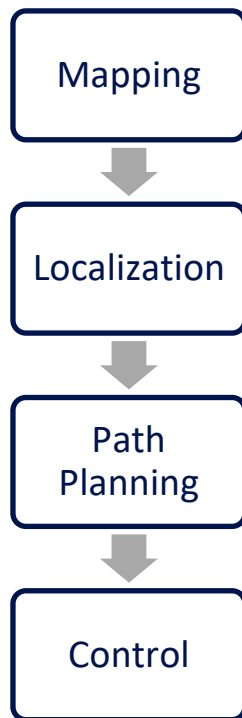
Outline

- Bayes Filter
- Kalman Filter
- Extended Kalman Filter (EKF)
- EKF SLAM

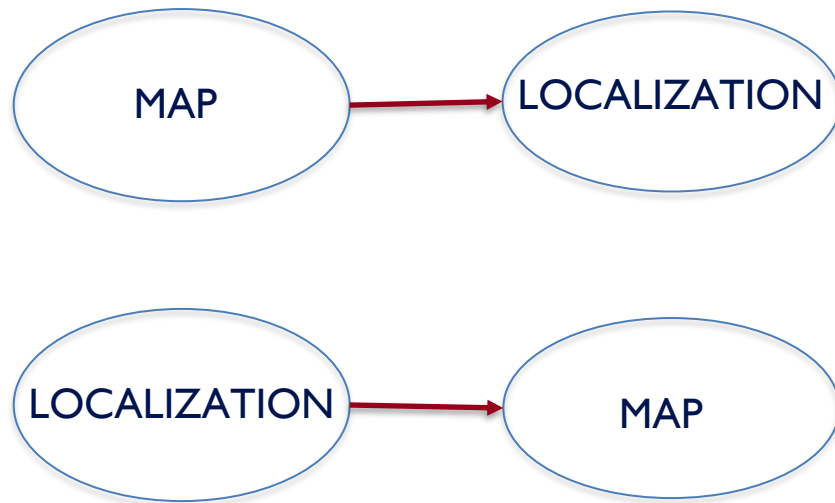
Localization

- “Using sensory information to locate the robot in its environment is the most fundamental problem to provide a mobile robot with autonomous capabilities.” [Cox'91]
- **Given**
 - Map of the environment
 - Data of sensor measurements
- **Wanted**
 - Estimation of the robot's position
 - Position tracking
 - Global localization

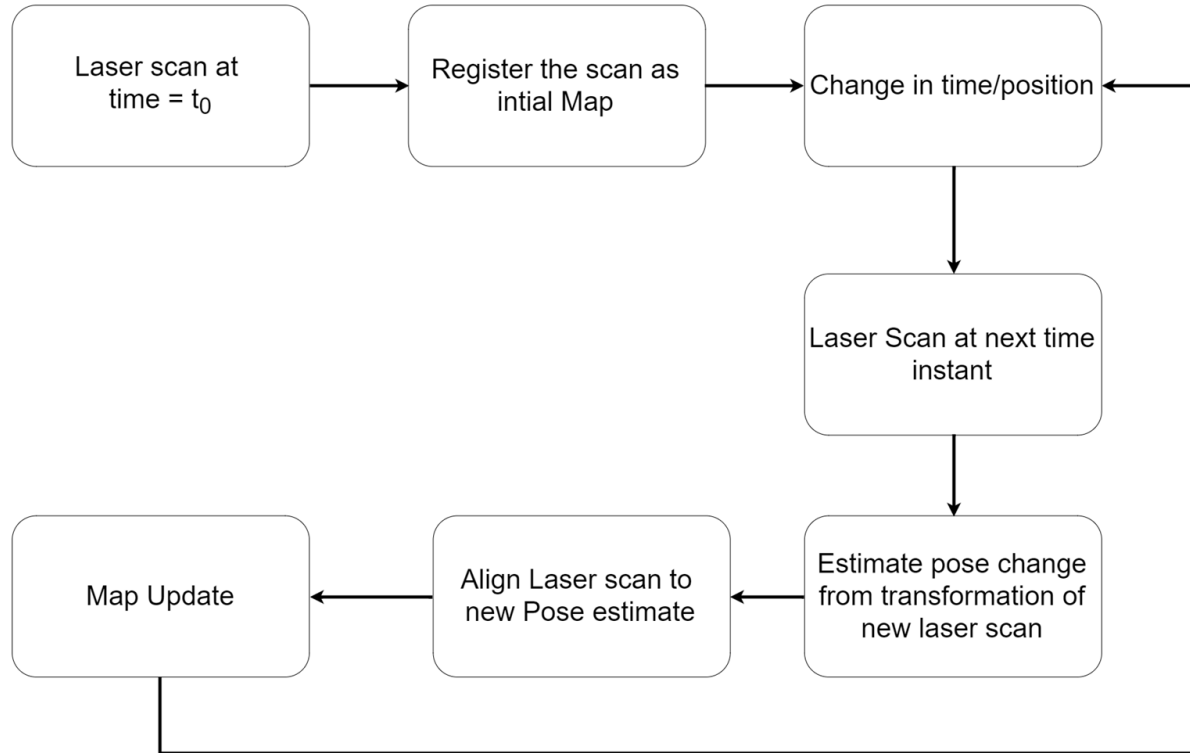
System Overview



SLAM : A Chicken-Egg problem

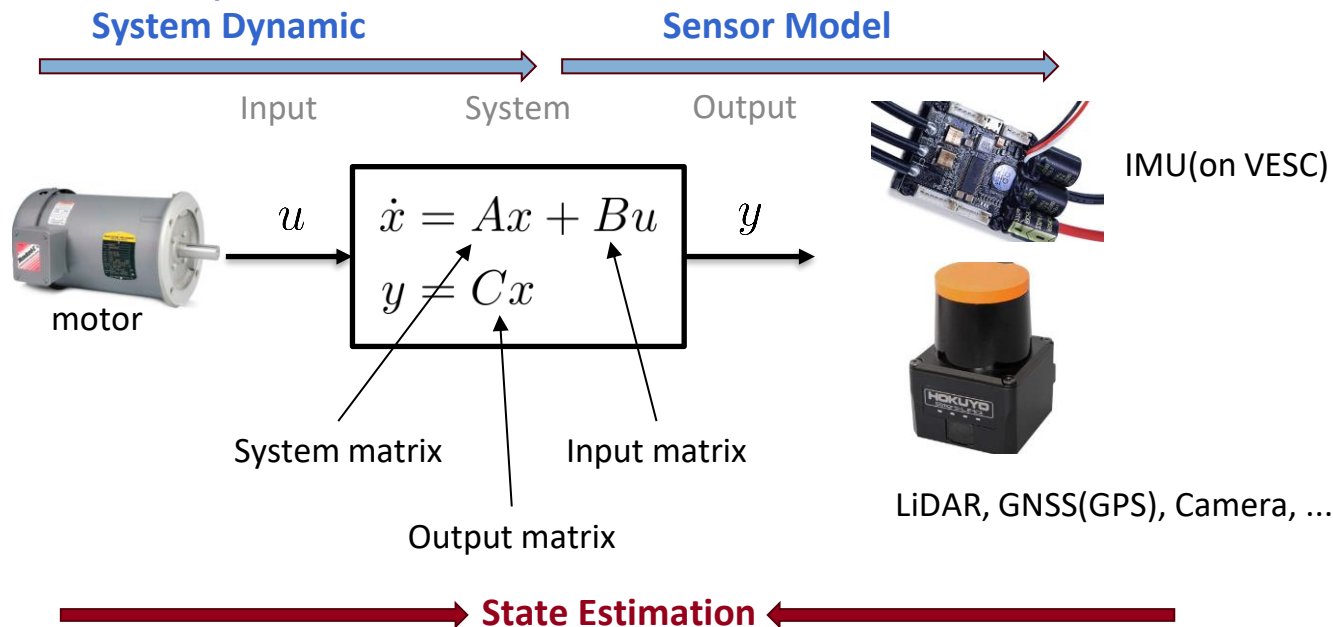


Overview of SLAM



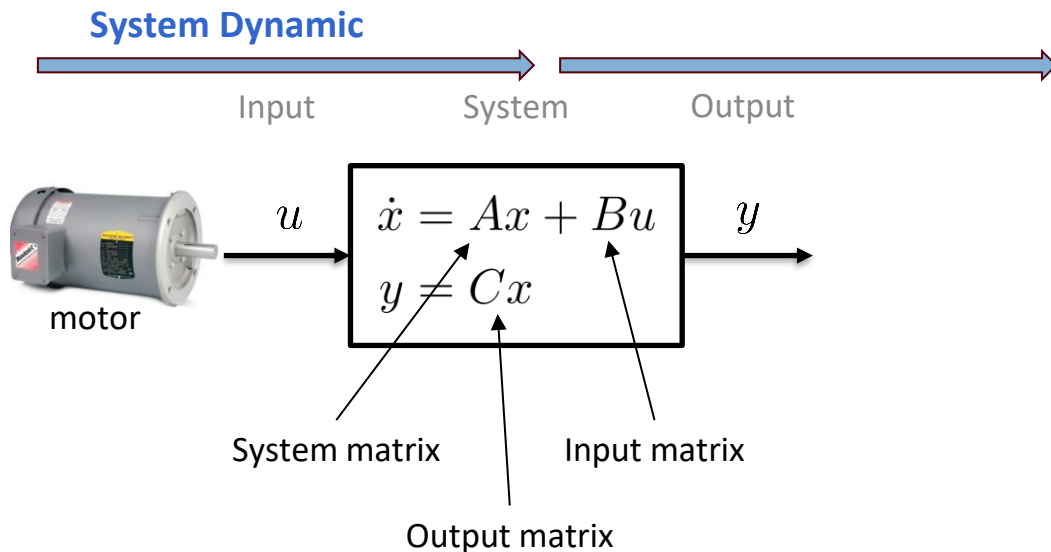
State Estimation

- Example of Vehicle Model
 - Linear system is assumed (Not in real world!)



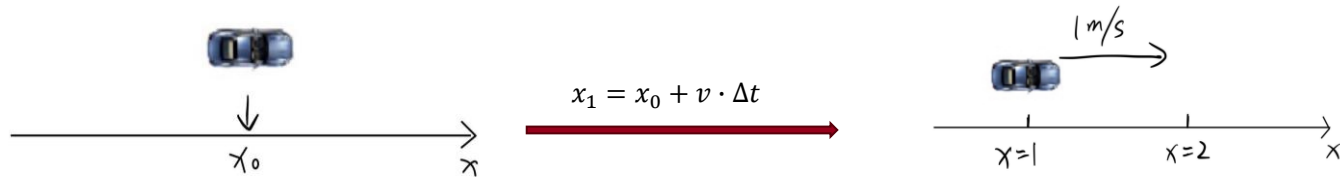
State Estimation

- What if we just consider system inputs to estimated vehicle's location?



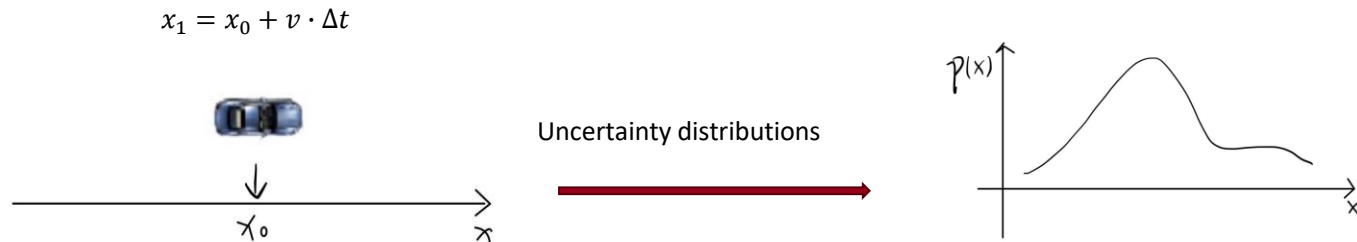
Dead Reckoning

- A primitive method used in navigation
- Calculate the current position of the car by using a previously position, and incorporate estimates of speed, heading (or direction) , and elapsed time



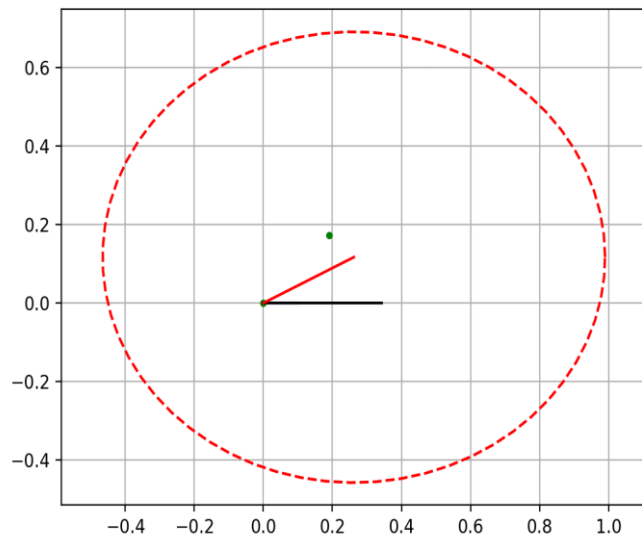
Dead Reckoning

- Unfortunately, ALL measurements have noises!
 - e.g., x_0, x_1, v , which means the position and velocity are not accurate.
 - After time elapsed, the error will be cumulated and causing drifting error in localization.
 - $x_0, x_1, x_2, \dots$



Drifting Example

- Results comparison of EKF estimation & Dead Reckoning & Ground Truth.



Blue line: True trajectory.

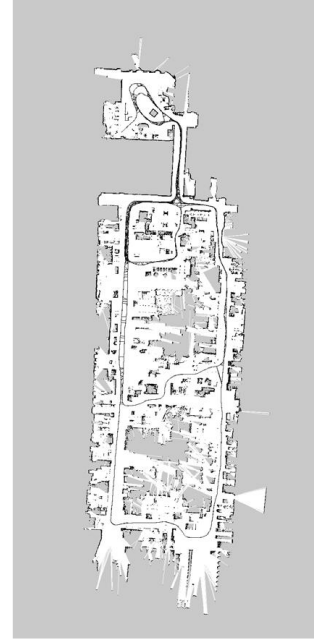
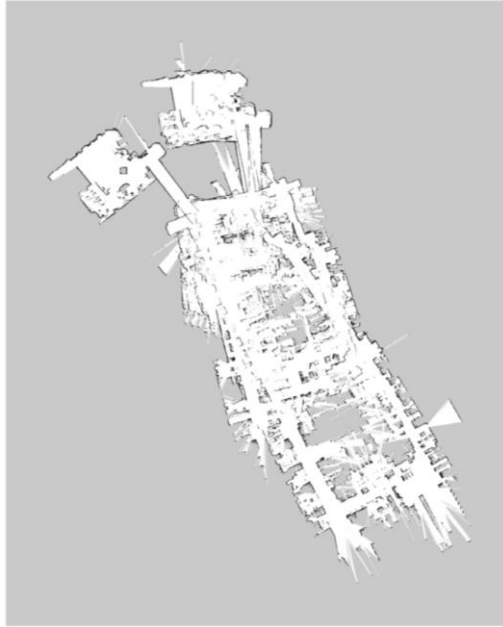
Black line: Dead reckoning trajectory,

Green points: positioning observation.

Red line: estimated trajectory with EKF.

Red ellipse: estimated covariance ellipse with EKF.

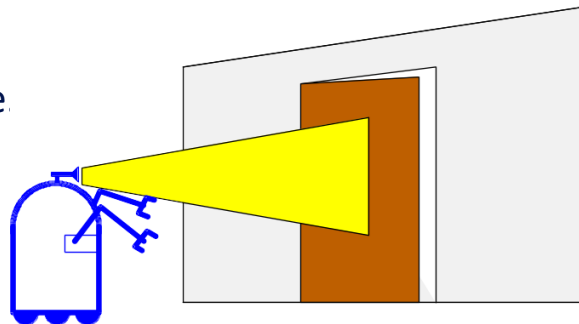
Drifting Example



Mapping result with dead reckoning (left) vs corrected estimation (right).

State Estimation Example

- Suppose a robot obtains a measurement z by its sensor.
- We want to estimate $p(\text{open} | z)$, with z known.
- Bayes Rule:
 - Bayes rule allows us to use causal knowledge
 - $p(\text{open} | z) = \frac{p(z | \text{open})p(\text{open})}{p(z)}$
- $p(\text{open} | z)$ is **diagnostic**, $p(z | \text{open})$ is **causal**.
 - Causal knowledge is easy to obtain.
- Assuming the sensor model:



Sensor model (likelihood):

- $p(z = \text{sense open} | \text{open}) = 0.6$
- $p(z = \text{sense open} | \neg \text{open}) = 0.3$

Prior knowledge(non-informative):

- $p(\text{open}) = p(\neg \text{open}) = 0.5$

State Estimation Example

- Update / Correction:

- Bayes Rule:

$$- \quad p(open|z) = \frac{p(z|open)p(open)}{p(z)} = \frac{p(z|open)p(open)}{p(z|open)p(open) + p(z|\neg open)p(\neg open)}$$

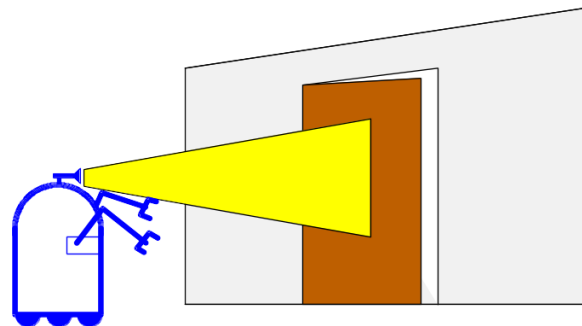
$$- \quad p(open|z) = \frac{0.6*0.5}{0.6*0.5 + 0.3*0.5} = 0.667$$

Sensor model (likelihood):

- $p(z = \text{sense open} | \text{open}) = 0.6$
- $p(z = \text{sense open} | \neg \text{open}) = 0.3$

Prior knowledge(non-informative):

- $p(open) = p(\neg open) = 0.5$



State Estimation Example

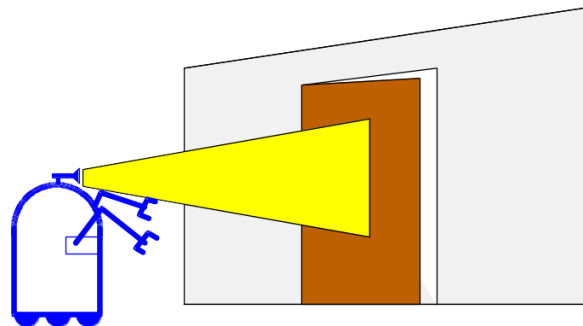
- The robot will receive new measurements continuously, $z_2, z_3, z_4, \dots, z_n$.
- How can we integrate $p(x|z_1, z_2, z_3, \dots, z_n)$?
- Markov assumption:
 - z_n is independent of $z_1, z_2, z_3, \dots, z_{n-1}$, if we know x .
 - The future is independent of the past if the present is known.

Sensor model (likelihood):

- $p(z = \text{sense open} \mid \text{open}) = 0.6$
- $p(z = \text{sense open} \mid \neg \text{open}) = 0.3$

Prior knowledge(non-informative):

- $p(\text{open}) = p(\neg \text{open}) = 0.5$



Recursive Bayesian Updating

- $p(x|z_1, z_2, z_3, \dots, z_n) = \frac{p(z_n|x, z_1, \dots, z_{n-1})p(x|z_1, \dots, z_{n-1})}{p(z_n|z_1, \dots, z_{n-1})}$

Note that

$$p(x|y, z) = \frac{p(y|x, z)p(x|z)}{p(y|z)}$$

- Markov assumption:

z_n is independent of $z_1, z_2, z_3, \dots, z_{n-1}$, if we know x .

- $$\begin{aligned} p(x|z_1, z_2, z_3, \dots, z_n) &= \frac{p(z_n|x)p(x|z_1, \dots, z_{n-1})}{p(z_n|z_1, \dots, z_{n-1})} \\ &= \eta_n p(z_n|x) p(x|z_1, \dots, z_{n-1}) = \eta_{n-1} p(z_{n-1}|x) p(x|z_1, \dots, z_{n-2}) \\ &= \eta_{1:n} \prod_{i=1}^n p(z_i|x) p(x) \end{aligned}$$

where $\eta_{1:n} = \eta_1 \eta_2 \dots \eta_n$.

Bayes Rule with More Conditions

$$p(x|z_1, z_2, z_3, \dots, z_n) = \frac{p(z_n|x, z_1, \dots, z_{n-1})p(x|z_1, \dots, z_{n-1})}{p(z_n|z_1, \dots, z_{n-1})}$$

When we combine the historical information: Recursive Bayes filter

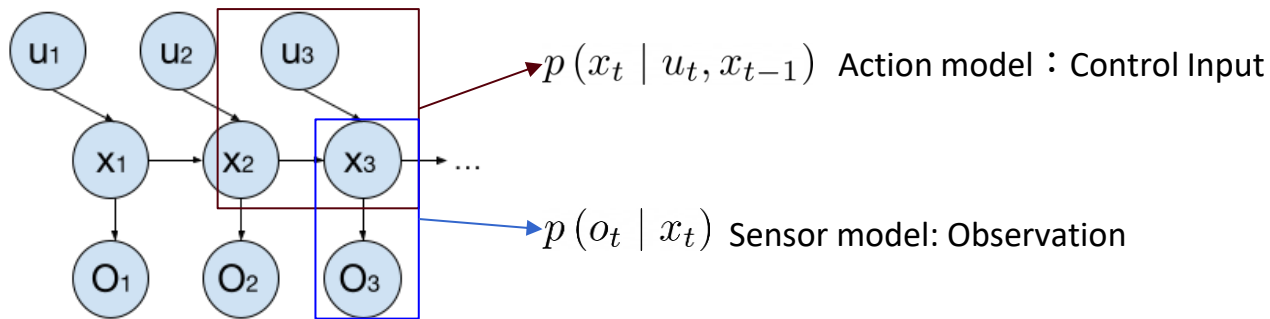


$$\text{Bel}(x_t) = P(x_t \mid o_t, u_t, o_{t-1}, u_{t-1}, \dots)$$

Diagram illustrating the components of the belief function $\text{Bel}(x_t)$:

- $\text{Bel}(x_t)$: Belief or posterior
- x_t : Robot state
- o_t : Observation
- u_t : Control Input
- o_{t-1} : History observation

Problem Setting: Hidden Markov Model



Markov Property: conditional independence

$$\mathbf{P}(O_t | x_1, \dots, x_t, u_1, \dots, u_t) = \mathbf{P}(O_t | x_t)$$

$$\mathbf{P}(x_t | x_1, \dots, x_{t-1}, u_1, \dots, u_{t-1}) = \mathbf{P}(x_t | x_{t-1}, u_t)$$

$$\text{Bel}(x_t) = P(x_t | o_t, u_t, o_{t-1}, u_{t-1}, \dots)$$

Bayes Filter (Recursive Bayes Estimation)

- Step1: Prediction with the control input (like in dead reckoning)

$$\begin{aligned}\overline{\text{bel}}(x_t) &= P(x_t \mid o_{1:t-1}, u_{1:t}) \\&= \int P(x_t \mid x_{t-1}, o_{1:t-1}, u_{1:t}) \boxed{P(x_{t-1} \mid o_{1:t-1}, u_{1:t})} d(x_{t-1}) \\&= \int P(x_t \mid x_{t-1}, o_{1:t-1}, u_{1:t}) \boxed{\text{bel}(x_{t-1})} d(x_{t-1}) \\&= \int P(x_t \mid x_{t-1}, u_t) \boxed{\text{bel}(x_{t-1})} d(x_{t-1})\end{aligned}$$

Prior (points to $\overline{\text{bel}}(x_t)$)

Law of Total Probability (points to the boxed term $P(x_{t-1} \mid o_{1:t-1}, u_{1:t})$)

Markov Property (points to the boxed term $\text{bel}(x_{t-1})$)

Bayes Filter

Step1: Prediction with the control input (like in dead reckoning)

$$\overline{\text{bel}}(x_t) = P(x_t | o_{1:t-1}, u_{1:t}) = \int P(x_t | x_{t-1}, u_t) \text{bel}(x_{t-1}) d(x_{t-1})$$

Step2: Correction with the observation

$$\text{bel}(x_t) = P(x_t | o_{1:t-1}, \boxed{o_t}, u_{1:t})$$
$$= \frac{P(o_t | o_{1:t-1}, x_t, u_{1:t}) P(x_t | o_{1:t-1}, u_{1:t})}{P(o_t | o_{1:t-1}, u_{1:t})}$$

Bayes Filter

Step1: Prediction with the control input (like in dead reckoning)

$$\overline{\text{bel}}(x_t) = P(x_t | o_{1:t-1}, u_{1:t}) = \int P(x_t | x_{t-1}, u_t) \text{bel}(x_{t-1}) d(x_{t-1})$$

← **Prior**
← **State transition(Action model)**

Step2: Correction with the observation

$$\begin{aligned} \text{bel}(x_t) &= P(x_t | o_{1:t-1}, o_t, u_{1:t}) \\ &= \frac{P(o_t | o_{1:t-1}, x_t, u_{1:t}) \boxed{P(x_t | o_{1:t-1}, u_{1:t})}}{P(o_t | o_{1:t-1}, u_{1:t})} \\ &= \frac{P(o_t | x_t) \boxed{P(x_t | o_{1:t-1}, u_{1:t})}}{\boxed{P(o_t | o_{1:t-1}, u_{1:t})}} \end{aligned}$$

← **Posterior**
← **Bayes Rule**
← **Prior**
← **Normalization term**

← **Markov Property**

Bayes Filter

Step1: Prediction with the control input

$$\overline{\text{bel}}(x_t) = P(x_t | o_{1:t-1}, u_{1:t}) = \int \boxed{P(x_t | x_{t-1}, u_t)} \boxed{\overline{\text{bel}}(x_{t-1})} d(x_{t-1})$$

Multiplication and integration of two probability distributions

Step2: Correction with the observation

$$\text{bel}(x_t) = \frac{P(o_t | x_t) \overline{\text{bel}}(x_t)}{P(o_t | o_{1:t-1}, u_{1:t})} = \boxed{\eta P(o_t | x_t)} \boxed{\overline{\text{bel}}(x_t)}$$

Bayes Filter

Algorithm 1 Bayes-filter

Require: Belief $bel(x_{t-1}) = p(x_{t-1} | z_{1:t-1}, u_{1:t-1})$, action u_t , measurement z_t ;

1: **for** all state variables **do**

2: $\overline{bel}(x_t) = \int p(x_t | x_{t-1}, u_t) bel(x_{t-1}) dx_{t-1}$ // Predict using action/control input u_t

3: $bel(x_t) = \eta p(z_t | x_t) \overline{bel}(x_t)$ // Update using perceptual data z_t

4: **return** $bel(x_t)$

Bayes Filter Examples

- **Linear**
 - Kalman Filter (KF), unimodal linear filter
 - Information Filter (IF), unimodal linear filter
- **Non-Linear**
 - Extended Kalman Filter (EKF): unimodal linear filter with the assumption of Gaussian noise
 - Particle Filter: multimodal non-linear filter

Kalman Filter

- The Kalman filter assumes a linear transition and observation model
- Zero mean Gaussian noise

$$x_t = A_t x_{t-1} + B_t u_t + \epsilon_t$$

$$z_t = C_t x_t + \delta_t$$

Kalman Filter Components

A_t Matrix $(n \times n)$ that describes how the state evolves from $t - 1$ to t without controls or noise.

B_t Matrix $(n \times l)$ that describes how the control u_t changes the state from $t - 1$ to t .

C_t Matrix $(k \times n)$ that describes how to map the state x_t to an observation z_t .

ϵ_t Random variables representing the process and measurement noise that are assumed to be independent and normally distributed with covariance R_t and Q_t respectively.

δ_t

Everything stays Gaussian

- Given an initial Gaussian belief, the belief stays Gaussian

$$\frac{\text{Gaussian}}{\text{Gaussian}} = \eta \frac{\text{Gaussian}}{\text{Gaussian}} \frac{\text{Gaussian}}{\text{Gaussian}}$$

$$\overline{\text{Gaussian}}(x_t) = \int \frac{\text{Gaussian}}{\text{Gaussian}} p(x_t \mid u_t, x_{t-1}) \frac{\text{Gaussian}}{\text{Gaussian}}(x_{t-1}) dx_{t-1}$$

- The product of two Gaussian is again a Gaussian
- We only need to show that $\overline{\text{Gaussian}}(x_t)$ is Gaussian so that $\text{Gaussian}(x_t)$ is Gaussian

Kalman Filter Algorithm

1: **Kalman_filter**($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):

2: $\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$

3: $\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$

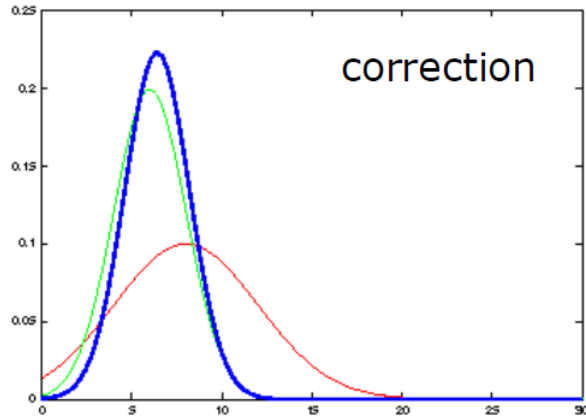
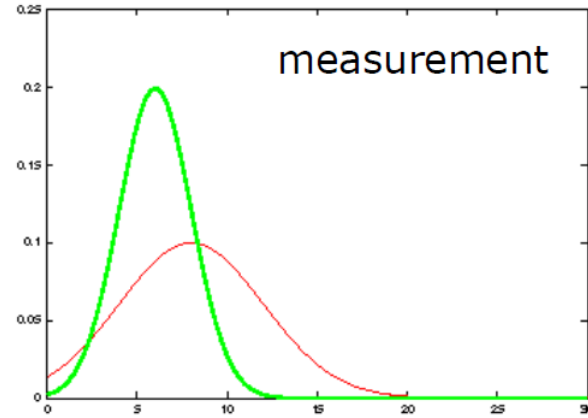
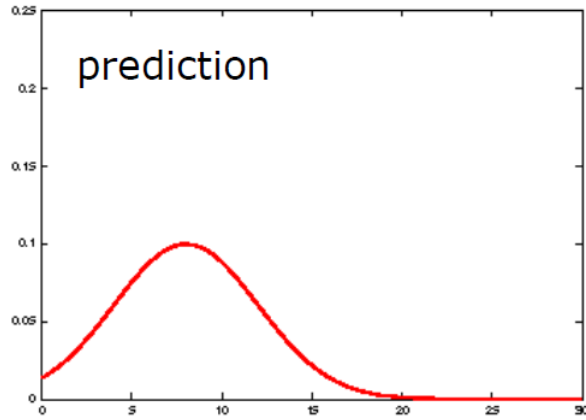
4: $K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$

5: $\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$

6: $\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$

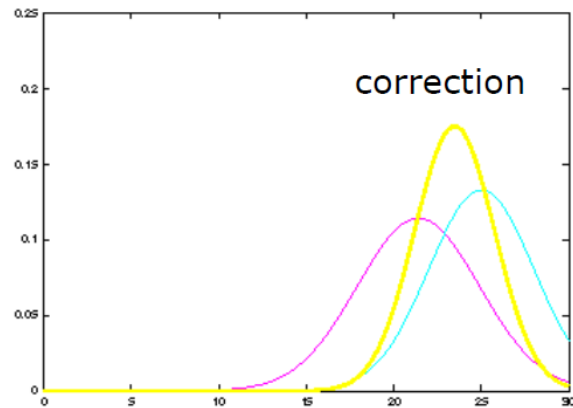
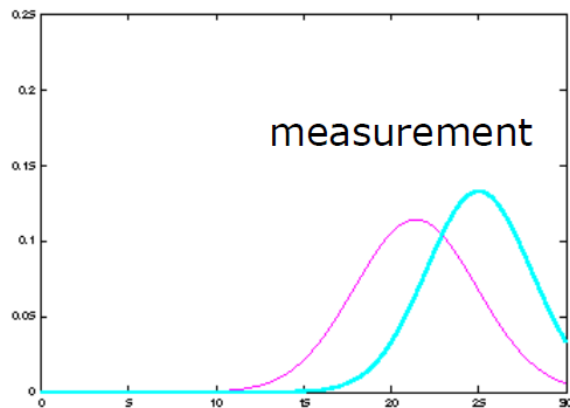
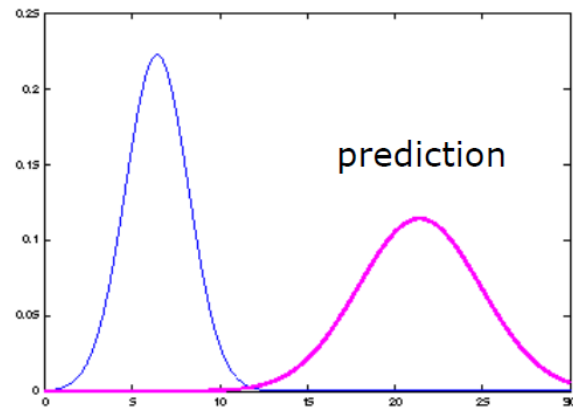
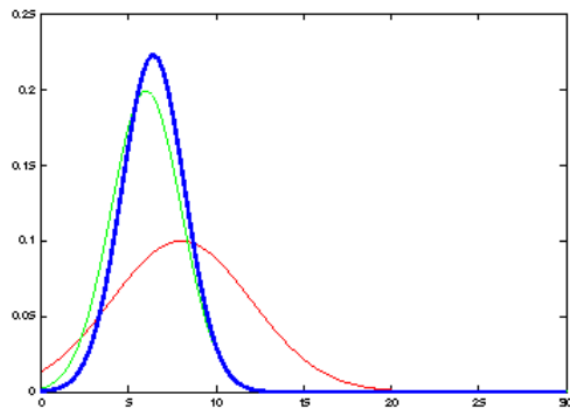
7: *return* μ_t, Σ_t

1D-Kalman Filter Example (1)



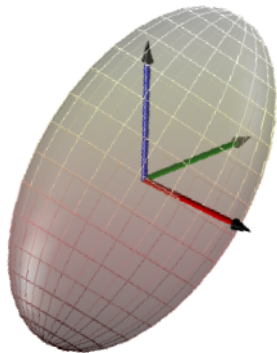
It's a weighted mean!

1D-Kalman Filter Example (2)



Kalman Filter Assumptions

- Gaussian distributions and noise
- Linear motion and observation model



$$x_t = A_t x_{t-1} + B_t u_t + \epsilon_t$$

$$z_t = C_t x_t + \delta_t$$

What if this is not the case?

Non-linear Dynamic Systems

- Most realistic problems (in robotics) involve nonlinear functions

~~$$x_t = A_t x_{t-1} + B_t u_t + \epsilon_t$$~~



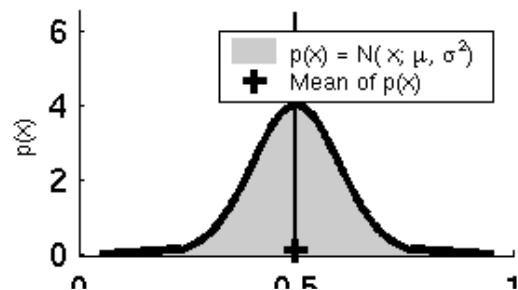
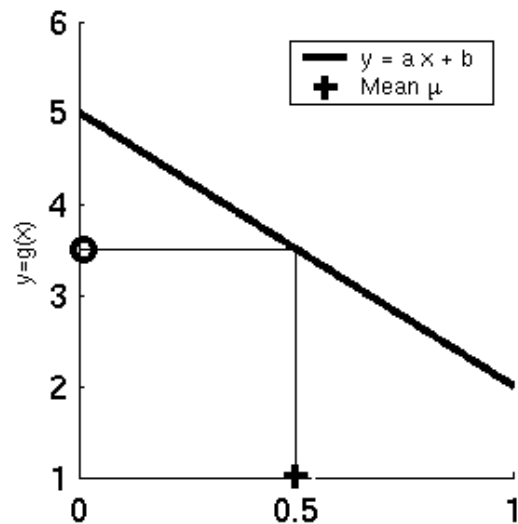
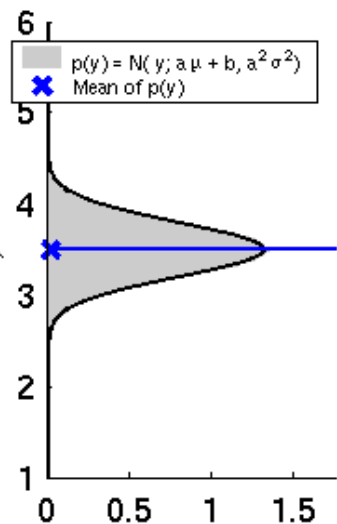
$$x_t = g(u_t, x_{t-1}) + \epsilon_t$$

~~$$z_t = C_t x_t + \delta_t$$~~

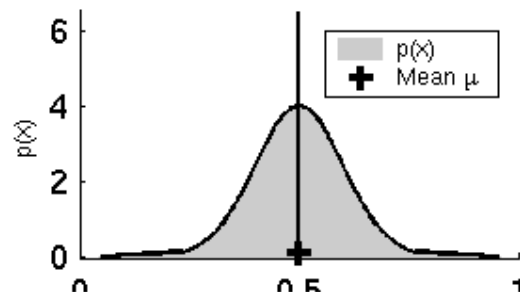
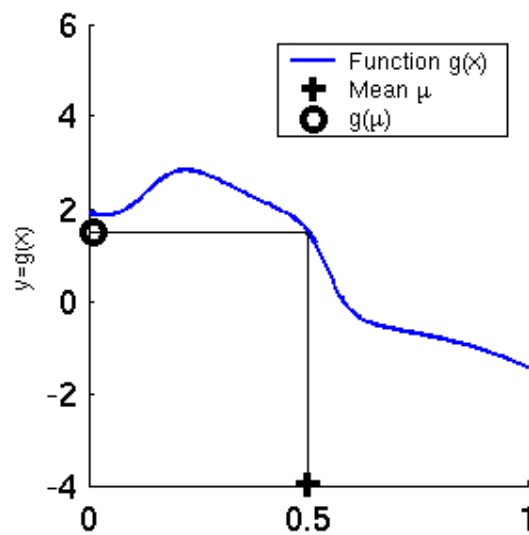
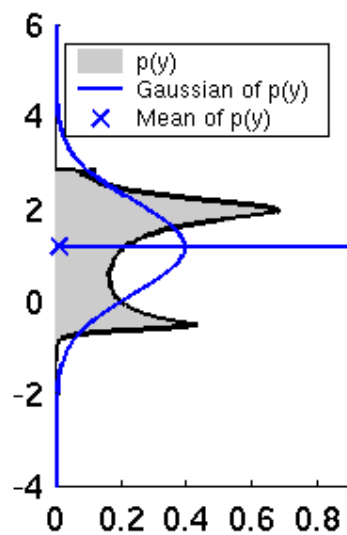


$$z_t = h(x_t) + \delta_t$$

Linear Assumption Revisited



Non-linear Function



Non-Gaussian Distributions

- The non-linear functions lead to non-Gaussian distributions
- Kalman filter is not applicable anymore!

What can be done to resolve this?

Local linearization!

Extended Kalman Filter (EKF) Linearization

- First-order Taylor Expansion

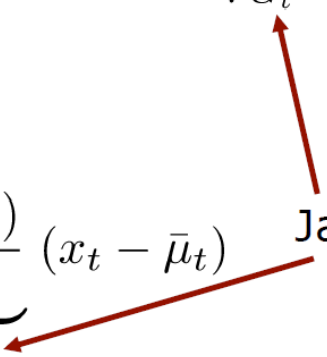
- Prediction:

$$g(u_t, x_{t-1}) \approx g(u_t, \mu_{t-1}) + \underbrace{\frac{\partial g(u_t, \mu_{t-1})}{\partial x_{t-1}}}_{=: G_t} (x_{t-1} - \mu_{t-1})$$

- Correction:

$$h(x_t) \approx h(\bar{\mu}_t) + \underbrace{\frac{\partial h(\bar{\mu}_t)}{\partial x_t}}_{=: H_t} (x_t - \bar{\mu}_t)$$

Jacobian matrices



Reminder: Jacobian Matrix

- It is a **non-square matrix** $m \times n$ in general
- Given a vector-valued function

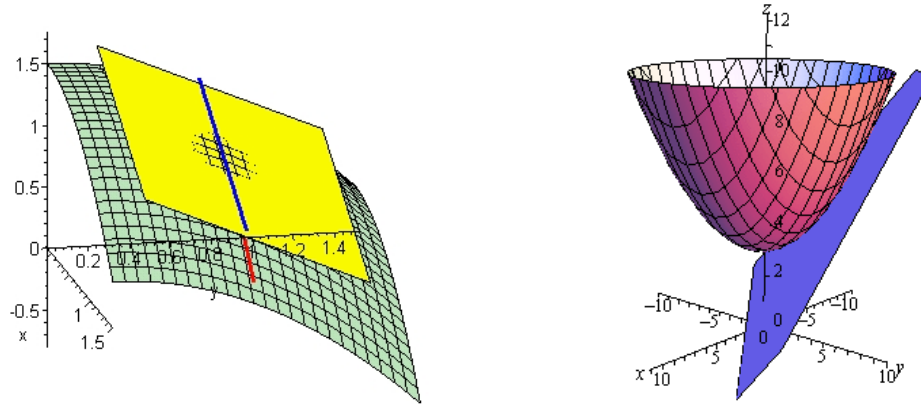
$$g(x) = \begin{pmatrix} g_1(x) \\ g_2(x) \\ \vdots \\ g_m(x) \end{pmatrix}$$

- The **Jacobian matrix** is defined as

$$G_x = \begin{pmatrix} \frac{\partial g_1}{\partial x_1} & \frac{\partial g_1}{\partial x_2} & \cdots & \frac{\partial g_1}{\partial x_n} \\ \frac{\partial g_2}{\partial x_1} & \frac{\partial g_2}{\partial x_2} & \cdots & \frac{\partial g_2}{\partial x_n} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\partial g_m}{\partial x_1} & \frac{\partial g_m}{\partial x_2} & \cdots & \frac{\partial g_m}{\partial x_n} \end{pmatrix}$$

Reminder: Jacobian Matrix

- It is the orientation of the tangent plane to the vector-valued function at a given point



- Generalizes the gradient of a scalar valued function

Extended Kalman Filter (EKF) Linearization

- First-order Taylor Expansion

- Prediction:

The local point (state) we consider

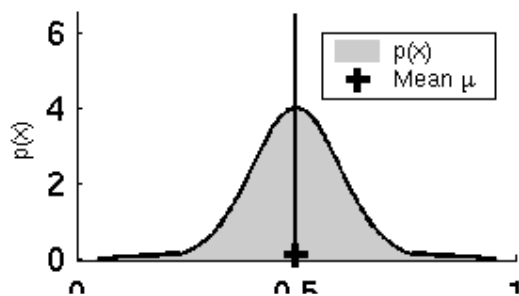
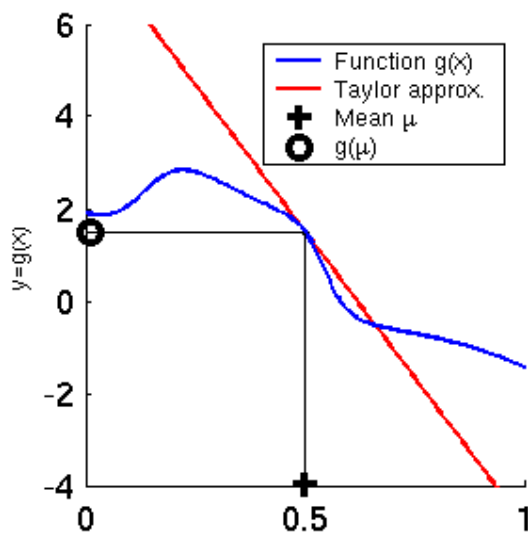
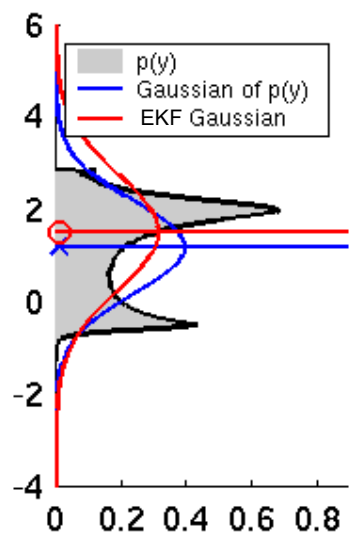
$$g(u_t, x_{t-1}) \approx g(u_t, \mu_{t-1}) + \underbrace{\frac{\partial g(u_t, \mu_{t-1})}{\partial x_{t-1}}}_{=: G_t} (x_{t-1} - \mu_{t-1})$$

- Correction:

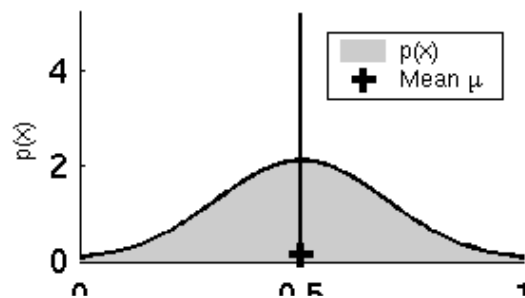
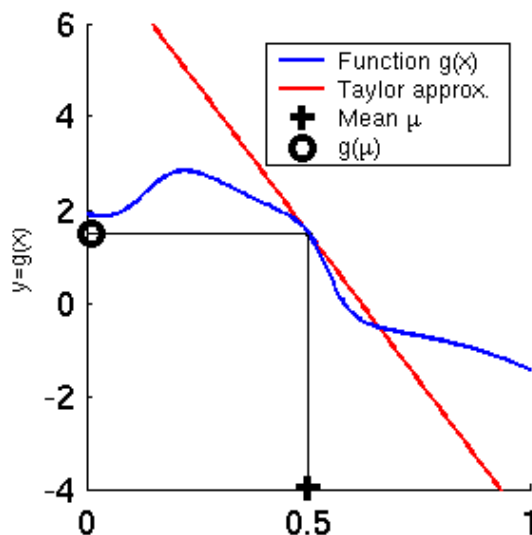
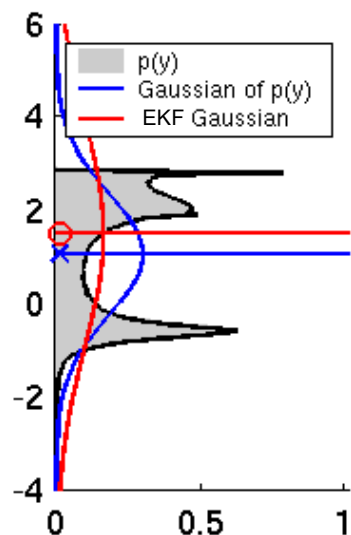
$$h(x_t) \approx h(\bar{\mu}_t) + \underbrace{\frac{\partial h(\bar{\mu}_t)}{\partial x_t}}_{=: H_t} (x_t - \bar{\mu}_t)$$

Linear functions!

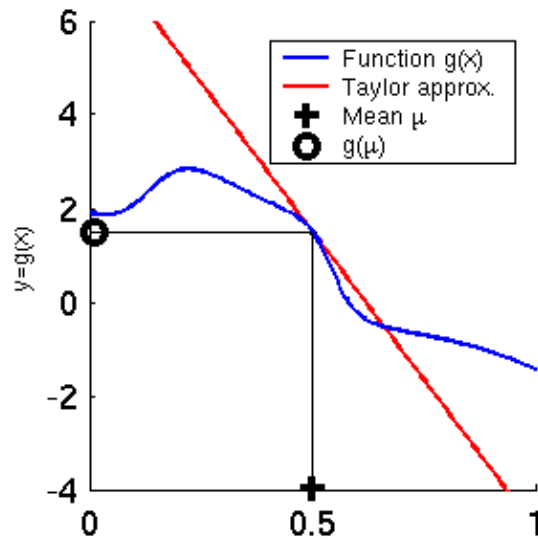
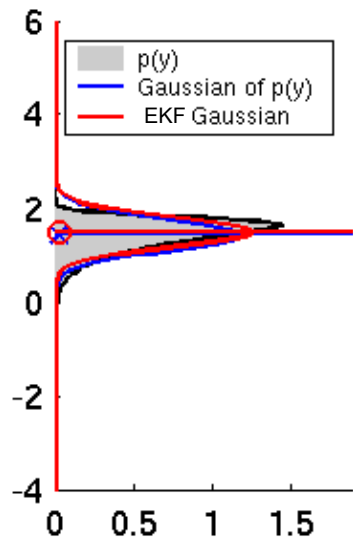
EKF Linearization (1)



EKF Linearization (2)

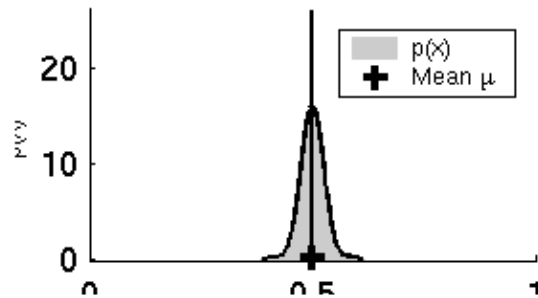


EKF Linearization (3)



Two key factors for achieving good approximation

- How **far** am I **away from the actual non-linear function?** (red vs blue)
- What is the **uncertainty of the input?**



Extended Kalman Filter (EKF)

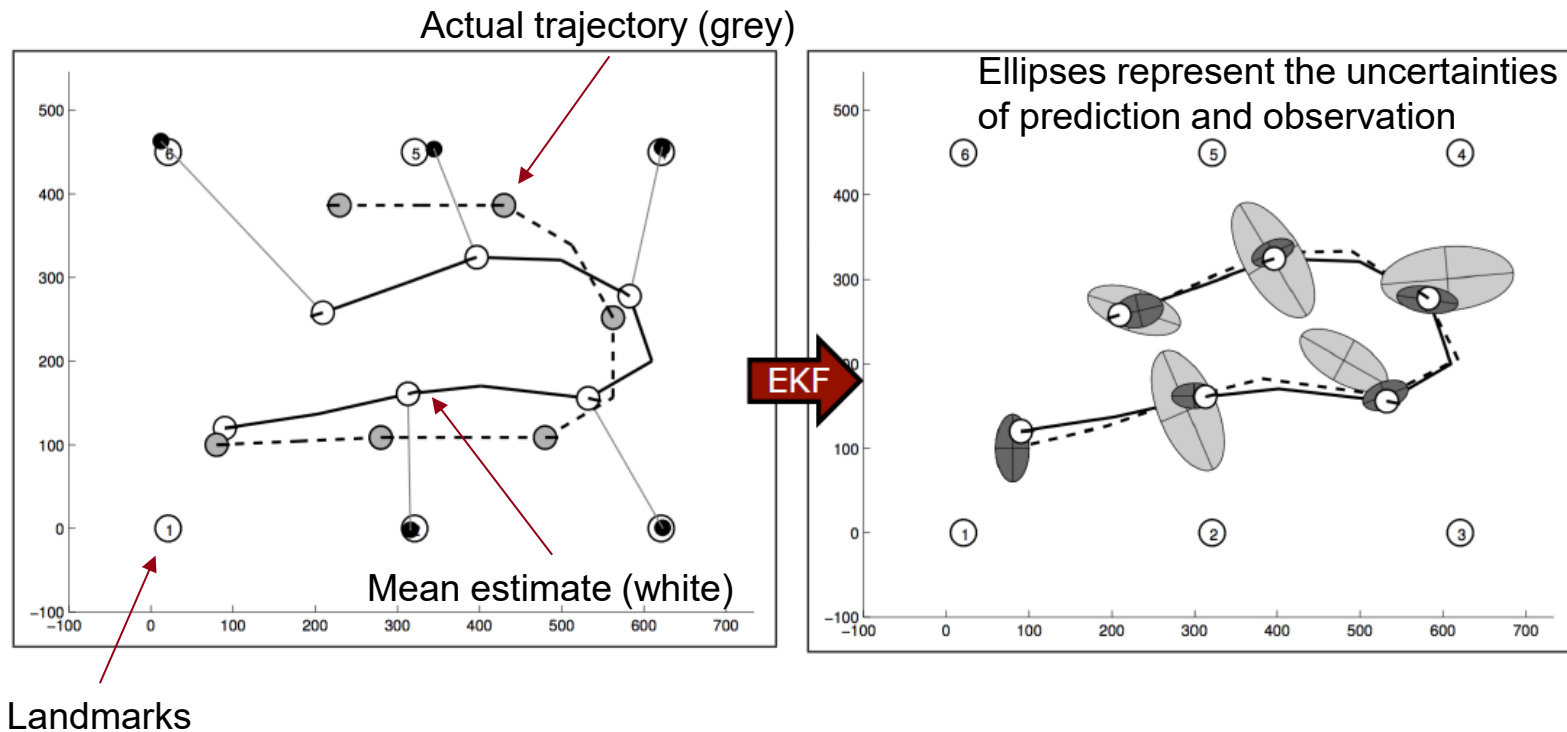
- 1: **Extended_Kalman_filter**($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):
- 2: $\bar{\mu}_t = \underline{g(u_t, \mu_{t-1})}$
- 3: $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$
- 4: $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$
- 5: $\mu_t = \bar{\mu}_t + K_t(z_t - \underline{h(\bar{\mu}_t)})$
- 6: $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$
- 7: *return* μ_t, Σ_t

$$A_t \leftrightarrow G_t$$

$$C_t \leftrightarrow H_t$$

KF vs. EKF

EKF Localization Example



weighted sum of predictions and observations

EKF Summary

- Extension of the Kalman filter
- One way to handle the non-linearities
- Performs local linearizations
- Works well in practice for moderate non-linearities
- Large uncertainty leads to increased approximation error

EKF SLAM

Practical example of EKF-SLAM --- Victoria Park Dataset.

- Sensor data from wheel encoder odometry, GPS, and laser scanner.
- A classical 2D state estimation with velocity-based motion model and landmark-based (trees as landmarks) observation.



Fig.1. Data Collection Vehicle



Fig.2. Data Collection Trajectory

EKF SLAM

- Application of the EKF to SLAM
- Estimate robot's pose and locations of landmarks in the environment
- Assumption: known correspondences
- State space (for the 2D plane) is

$$x_t = \left(\underbrace{x, y, \theta}_{\text{robot's pose}}, \underbrace{m_{1,x}, m_{1,y}}_{\text{landmark 1}}, \dots, \underbrace{m_{n,x}, m_{n,y}}_{\text{landmark n}} \right)^T$$

EKF SLAM: State Representation

- Map with n landmarks: $(3+2n)$ -dimensional Gaussian
- Belief is represented by

$$\underbrace{\begin{pmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{x_t x_t} & \Sigma_{x_t m_1} & \dots & \Sigma_{x_t m_n} \\ \Sigma_{m_1 x_t} & \Sigma_{m_1 m_1} & \dots & \Sigma_{m_1 m_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_n x_t} & \Sigma_{m_n m_1} & \dots & \Sigma_{m_n m_n} \end{pmatrix}}_{\Sigma} \quad \xrightarrow{\text{Simplify}} \quad \underbrace{\begin{pmatrix} x \\ m \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{xx} & \Sigma_{xm} \\ \Sigma_{mx} & \Sigma_{mm} \end{pmatrix}}_{\Sigma}$$

EKF SLAM: Filter Cycle

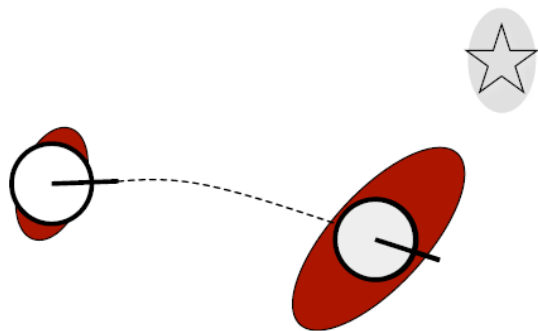
- State prediction
- Measurement prediction
- Measurement
- Data association
- Update

EKF SLAM: Initial State



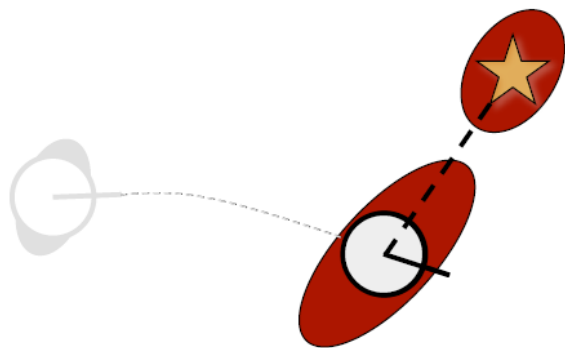
$$\underbrace{\begin{pmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{x_t x_t} & \Sigma_{x_t m_1} & \cdots & \Sigma_{x_t m_n} \\ \Sigma_{m_1 x_t} & \Sigma_{m_1 m_1} & \cdots & \Sigma_{m_1 m_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_n x_t} & \Sigma_{m_n m_1} & \cdots & \Sigma_{m_n m_n} \end{pmatrix}}_{\Sigma}$$

EKF SLAM: Predicted Motion



$$\underbrace{\begin{pmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{x_t x_t} & \Sigma_{x_t m_1} & \dots & \Sigma_{x_t m_n} \\ \Sigma_{m_1 x_t} & \Sigma_{m_1 m_1} & \dots & \Sigma_{m_1 m_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_n x_t} & \Sigma_{m_n m_1} & \dots & \Sigma_{m_n m_n} \end{pmatrix}}_{\Sigma}$$

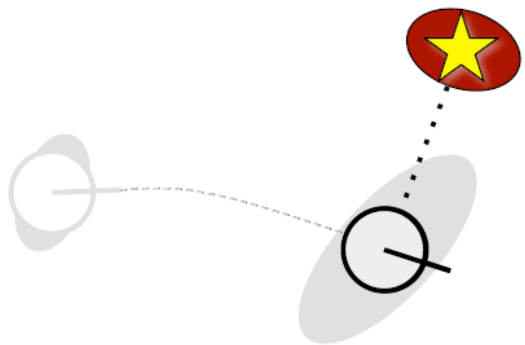
EKF SLAM: Predicted Measurement



No matrix update

$$\underbrace{\begin{pmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{x_t x_t} & \Sigma_{x_t m_1} & \dots & \Sigma_{x_t m_n} \\ \Sigma_{m_1 x_t} & \Sigma_{m_1 m_1} & \dots & \Sigma_{m_1 m_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_n x_t} & \Sigma_{m_n m_1} & \dots & \Sigma_{m_n m_n} \end{pmatrix}}_{\Sigma}$$

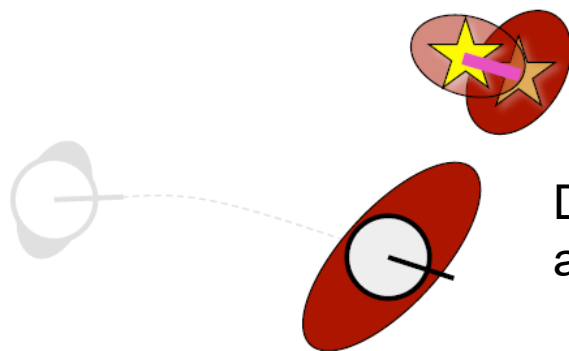
EKF SLAM: Obtained Measurement



No matrix update

$$\underbrace{\begin{pmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{x_t x_t} & \Sigma_{x_t m_1} & \dots & \Sigma_{x_t m_n} \\ \Sigma_{m_1 x_t} & \Sigma_{m_1 m_1} & \dots & \Sigma_{m_1 m_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_n x_t} & \Sigma_{m_n m_1} & \dots & \Sigma_{m_n m_n} \end{pmatrix}}_{\Sigma}$$

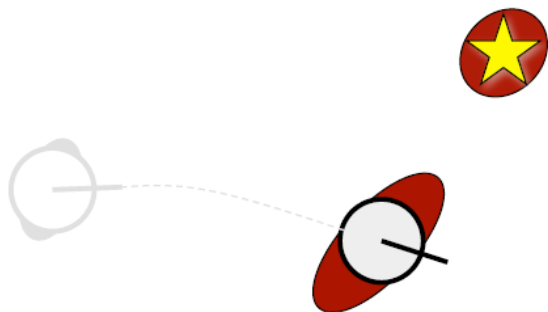
EKF SLAM: Data Association



Difference between **predicted observation** $h(x)$,
and **actual observation** z

$$\underbrace{\begin{pmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{x_t x_t} & \Sigma_{x_t m_1} & \dots & \Sigma_{x_t m_n} \\ \Sigma_{m_1 x_t} & \Sigma_{m_1 m_1} & \dots & \Sigma_{m_1 m_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_n x_t} & \Sigma_{m_n m_1} & \dots & \Sigma_{m_n m_n} \end{pmatrix}}_{\Sigma}$$

EKF SLAM: Update Step



- Robot's pose got updated
- Landmarks' locations got updated

$$\underbrace{\begin{pmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{x_t x_t} & \Sigma_{x_t m_1} & \dots & \Sigma_{x_t m_n} \\ \Sigma_{m_1 x_t} & \Sigma_{m_1 m_1} & \dots & \Sigma_{m_1 m_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_n x_t} & \Sigma_{m_n m_1} & \dots & \Sigma_{m_n m_n} \end{pmatrix}}_{\Sigma}$$

EKF SLAM: Concrete Example

Setup

- Platform moves in the 2D plane
- Velocity-based motion model
- Observation of point landmarks
- Range-bearing sensor
- Known data association
- Known number of landmarks

Initialization

- Platform starts in its own reference frame (all landmarks unknown)
- $2N+3$ dimensions

$$\mu_0 = (0 \ 0 \ 0 \ \dots \ 0)^T$$

$$\Sigma_0 = \begin{pmatrix} 0 & 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & \infty & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \infty \end{pmatrix}$$

Prediction Step (Motion)

- Goal: Update state space based on the motion
- Motion in the plane

$$\begin{pmatrix} x' \\ y' \\ \theta' \end{pmatrix} = \begin{pmatrix} x \\ y \\ \theta \end{pmatrix} + \underbrace{\begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix}}_{g_{x,y,\theta}(u_t, (x,y,\theta)^T)}$$

translation velocity rotation velocity

- How to map that to the $2N+3$ dim state space used in the EKF?

Update the State Space

- From the motion in the plane

$$\begin{pmatrix} x' \\ y' \\ \theta' \end{pmatrix} = \begin{pmatrix} x \\ y \\ \theta \end{pmatrix} + \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix}$$

- to the $2N+3$ dimensional space

$$\begin{pmatrix} x' \\ y' \\ \theta' \\ \vdots \end{pmatrix} = \underbrace{\begin{pmatrix} x \\ y \\ \theta \\ \vdots \end{pmatrix} + \underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \dots 0 \\ 0 & 1 & 0 & 0 \dots 0 \\ 0 & 0 & 1 & \underbrace{0 \dots 0}_{2Ncols} \end{pmatrix}^T}_{F_x^T} \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix} \underbrace{\hspace{10em}}_{g(u_t, x_t)}$$

EKF Algorithm

- 1: **Extended_Kalman_filter**($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):
- 2: ~~$\bar{\mu}_t = g(u_t, \mu_{t-1})$~~ **DONE**
- 3: $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$

- 4: $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$
- 5: $\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t))$
- 6: $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$
- 7: *return* μ_t, Σ_t

Update Covariance

- The function g only affects the motion and **not** the landmarks

Jacobian of the motion (3x3)

$$G_t = \begin{pmatrix} G_t^x & 0 \\ 0 & I \end{pmatrix}$$

Identity (2N x 2N)

Jacobian of the motion

$$\begin{aligned} G_t^x &= \frac{\partial}{\partial(x, y, \theta)^T} \left[\begin{pmatrix} x \\ y \\ \theta \end{pmatrix} + \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix} \right] \\ &= I + \frac{\partial}{\partial(x, y, \theta)^T} \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix} \\ &= I + \begin{pmatrix} 0 & 0 & -\frac{v_t}{\omega_t} \cos \theta + \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ 0 & 0 & -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ 0 & 0 & 0 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 & -\frac{v_t}{\omega_t} \cos \theta + \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ 0 & 1 & -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ 0 & 0 & 1 \end{pmatrix} \end{aligned}$$

This leads to the Update

1: **Extended_Kalman_filter**($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):

2: ~~$\bar{\mu}_t = g(u_t, \mu_{t-1})$~~ **Apply & DONE**

3:  $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$


$$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$$

$$= \begin{pmatrix} G_t^x & 0 \\ 0 & I \end{pmatrix} \begin{pmatrix} \Sigma_{xx} & \Sigma_{xm} \\ \Sigma_{mx} & \Sigma_{mm} \end{pmatrix} \begin{pmatrix} (G_t^x)^T & 0 \\ 0 & I \end{pmatrix} + R_t$$

$$= \begin{pmatrix} G_t^x \Sigma_{xx} (G_t^x)^T & G_t^x \Sigma_{xm} \\ (G_t^x \Sigma_{xm})^T & \Sigma_{mm} \end{pmatrix} + R_t$$

Uncertainties of landmarks
remain unchanged

EKF Algorithm

- 1: **Extended_Kalman_filter**($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):
- 2: ~~$\bar{\mu}_t = g(u_t, \mu_{t-1})$~~ **DONE**
- 3: ~~$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$~~ **DONE**
- 4: $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$
- 5: $\mu_t = \bar{\mu}_t + K_t(z_t - h(\bar{\mu}_t))$
- 6: $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$
- 7: *return* μ_t, Σ_t

EKF SLAM: Prediction Step

EKF_SLAM_Prediction($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t, c_t, R_t$):

2: $F_x = \begin{pmatrix} 1 & 0 & 0 & 0 \cdots 0 \\ 0 & 1 & 0 & 0 \cdots 0 \\ 0 & 0 & 1 & 0 \cdots 0 \end{pmatrix}$ A matrix that maps from 3-dim space to $2n+3$ dim space

3: $\bar{\mu}_t = \mu_{t-1} + F_x^T \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \mu_{t-1, \theta} + \frac{v_t}{\omega_t} \sin(\mu_{t-1, \theta} + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \mu_{t-1, \theta} - \frac{v_t}{\omega_t} \cos(\mu_{t-1, \theta} + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix}$

4: $G_t = I + F_x^T \begin{pmatrix} 0 & 0 & -\frac{v_t}{\omega_t} \cos \mu_{t-1, \theta} + \frac{v_t}{\omega_t} \cos(\mu_{t-1, \theta} + \omega_t \Delta t) \\ 0 & 0 & -\frac{v_t}{\omega_t} \sin \mu_{t-1, \theta} + \frac{v_t}{\omega_t} \sin(\mu_{t-1, \theta} + \omega_t \Delta t) \\ 0 & 0 & 0 \end{pmatrix} F_x$

5: $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + \underbrace{F_x^T R_t^x F_x}_{R_t}$

EKF Algorithm

- 1: **Extended_Kalman_filter**($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):
- 2: ~~$\bar{\mu}_t = g(u_t, \mu_{t-1})$~~ **DONE**
- 3: ~~$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$~~ **Apply & DONE**
- 4:  $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$
- 5: $\mu_t = \bar{\mu}_t + K_t(z_t - h(\bar{\mu}_t))$
- 6: $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$
- 7: *return* μ_t, Σ_t

EKF SLAM: Correction Step

- Assumption: **Known data association**
- $c_t^i = j$ measurement at time t observes the landmark with index j (**correspondence matrix**)
- Initialize landmark if unobserved
- Compute the expected observation
- Compute the Jacobian of h
- Proceed with computing the Kalman gain

Range-Bearing Observation

- Range-Bearing observation $z_t^i = (r_t^i, \phi_t^i)^T$
- If landmark has not been observed, we can initialize it with:

$$\begin{pmatrix} \bar{\mu}_{j,x} \\ \bar{\mu}_{j,y} \end{pmatrix} = \begin{pmatrix} \bar{\mu}_{t,x} \\ \bar{\mu}_{t,y} \end{pmatrix} + \begin{pmatrix} r_t^i \cos(\phi_t^i + \bar{\mu}_{t,\theta}) \\ r_t^i \sin(\phi_t^i + \bar{\mu}_{t,\theta}) \end{pmatrix}$$

location of
landmark j

estimated
location of
the platform

relative
measurement

Expected Observation: $h(x)$

- Compute expected observation according to the current estimate

$$\delta = \begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} = \begin{pmatrix} \bar{\mu}_{j,x} - \bar{\mu}_{t,x} \\ \bar{\mu}_{j,y} - \bar{\mu}_{t,y} \end{pmatrix}$$

$$q = \delta^T \delta \quad \text{Squared distance}$$

$$\begin{aligned} \hat{z}_t^i &= \begin{pmatrix} \sqrt{q} \\ \text{atan2}(\delta_y, \delta_x) - \bar{\mu}_{t,\theta} \end{pmatrix} \\ &= h(\bar{\mu}_t) \end{aligned}$$

Jacobian for the Observation

- Based on
$$\delta = \begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} = \begin{pmatrix} \bar{\mu}_{j,x} - \bar{\mu}_{t,x} \\ \bar{\mu}_{j,y} - \bar{\mu}_{t,y} \end{pmatrix}$$
$$q = \delta^T \delta$$
$$\hat{z}_t^i = \begin{pmatrix} \sqrt{q} \\ \text{atan2}(\delta_y, \delta_x) - \bar{\mu}_{t,\theta} \end{pmatrix}$$

- Compute the Jacobian

low-dim space 

$$\begin{aligned} {}^{\text{low}} H_t^i &= \frac{\partial h(\bar{\mu}_t)}{\partial \bar{\mu}_t} \\ (x, y, \theta, m_{j,x}, m_{j,y}) &= \begin{pmatrix} \frac{\partial \sqrt{q}}{\partial x} & \frac{\partial \sqrt{q}}{\partial y} & \dots \\ \frac{\partial \text{atan2}(\dots)}{\partial x} & \frac{\partial \text{atan2}(\dots)}{\partial y} & \dots \end{pmatrix} \end{aligned}$$

The First Component

- Based on
$$\delta = \begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} = \begin{pmatrix} \bar{\mu}_{j,x} - \bar{\mu}_{t,x} \\ \bar{\mu}_{j,y} - \bar{\mu}_{t,y} \end{pmatrix}$$
$$q = \delta^T \delta$$
$$\hat{z}_t^i = \begin{pmatrix} \sqrt{q} \\ \text{atan2}(\delta_y, \delta_x) - \bar{\mu}_{t,\theta} \end{pmatrix}$$
- We obtain (by applying the chain rule)

$$\begin{aligned} \frac{\partial \sqrt{q}}{\partial x} &= \frac{1}{2} \frac{1}{\sqrt{q}} 2 \delta_x (-1) \\ &= \frac{1}{q} (-\sqrt{q} \delta_x) \end{aligned}$$

Jacobian for the Observation

- Based on
$$\delta = \begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} = \begin{pmatrix} \bar{\mu}_{j,x} - \bar{\mu}_{t,x} \\ \bar{\mu}_{j,y} - \bar{\mu}_{t,y} \end{pmatrix}$$
$$q = \delta^T \delta$$
$$\hat{z}_t^i = \begin{pmatrix} \sqrt{q} \\ \text{atan2}(\delta_y, \delta_x) - \bar{\mu}_{t,\theta} \end{pmatrix}$$

- Compute the Jacobian

$$\begin{aligned} {}^{\text{low}} H_t^i &= \frac{\partial h(\bar{\mu}_t)}{\partial \bar{\mu}_t} \\ &= \frac{1}{q} \begin{pmatrix} -\sqrt{q}\delta_x & -\sqrt{q}\delta_y & 0 & +\sqrt{q}\delta_x & \sqrt{q}\delta_y \\ \delta_y & -\delta_x & -q & -\delta_y & \delta_x \end{pmatrix} \end{aligned}$$

Jacobian for the Observation

- map the computed Jacobian to the high dimensional space

$$H_t^i = {}^{\text{low}} H_t^i F_{x,j}$$


 $F_{x,j} =$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \dots 0 & 0 & 0 & 0 \dots 0 \\ 0 & 1 & 0 & 0 \dots 0 & 0 & 0 & 0 \dots 0 \\ 0 & 0 & 1 & 0 \dots 0 & 0 & 0 & 0 \dots 0 \\ 0 & 0 & 0 & 0 \dots 0 & \boxed{1} & 0 & 0 \dots 0 \\ 0 & 0 & 0 & \underbrace{0 \dots 0}_{2j-2} & 0 & 1 & \underbrace{0 \dots 0}_{2N-2j} \end{pmatrix}$$



Update the j -th landmark

EKF Algorithm

- 1: **Extended_Kalman_filter**($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):
- 2: ~~$\bar{\mu}_t = g(u_t, \mu_{t-1})$~~ **DONE**
- 3: ~~$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$~~ **DONE**
- 4: ~~$K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$~~ **Apply & DONE**
- 5: ~~$\mu_t = \bar{\mu}_t + K_t(z_t - h(\bar{\mu}_t))$~~ **Apply & DONE**
- 6: ~~$\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$~~ **Apply & DONE**
- 7:  **return** μ_t, Σ_t

Implementation Notes

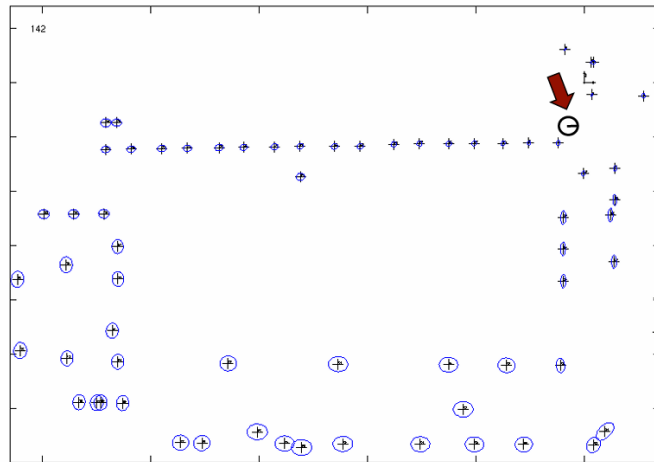
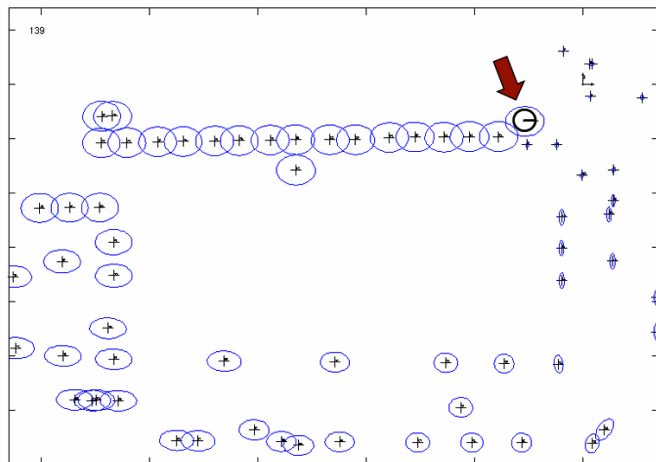
- Measurement update in a single step requires only one full belief update
- Always normalize the angular components $(-\pi, \pi)$
- You may not need to create the F matrices explicitly (e.g., in Matlab)

Loop Closing

- Loop closing means revisiting (and
- recognizing) an already mapped area
- Data association under
 - high ambiguity
 - possible environment symmetries
- Uncertainties **collapse** after a loop closure (whether the closure was correct or not)

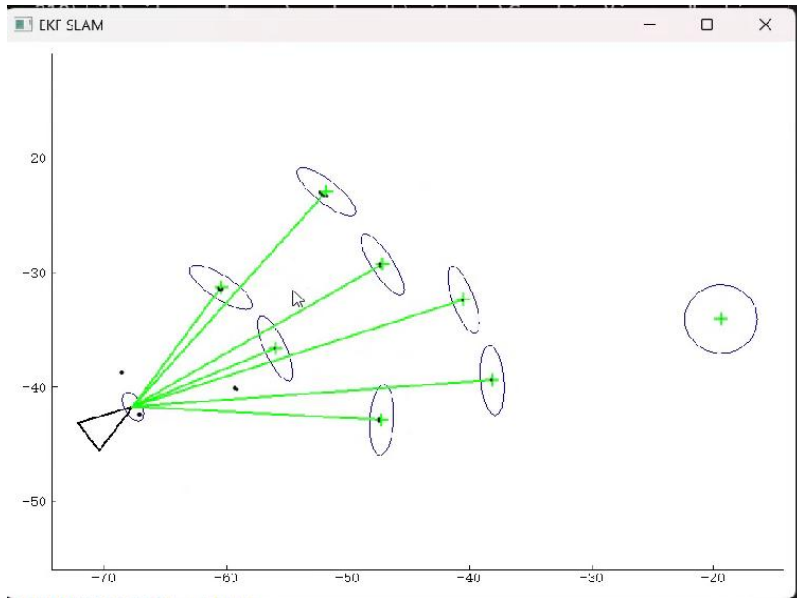
Loop Closure in SLAM

- Loop closing reduces the uncertainty in robot and landmark estimates
- This can be exploited when exploring an environment for the sake of better (e.g. more accurate) maps
- Wrong loop closures lead to filter divergence



EKF SLAM

- Coding Demo:
 - Landmarks sensed are shown with green cross marks and uncertainty ellipsoids
 - Every time the landmark is sensed, its uncertainty will decrease.



EKF SLAM Complexity

- Cubic complexity w.r.t. the measurement dimensionality
- Cost per step: **dominated by the number of landmarks: $O(n^2)$**
- Memory consumption: $O(n^2)$
- The EKF becomes **computationally intractable for large maps!**

EKF SLAM Summary

- Using the EKF to estimate pose and map in an SLAM fashion
- The first probabilistic SLAM approaches used the EKF
- Successful in **medium-scale scenes**
- Approximations exist to reduce the computational complexity
- Today's mainly used for **short-term estimates**

Questions?